Project 15 (Real-World Example) — MeetingMind-Transcript-To-Decisions-Actions-And-Summary

Type: Multi-agent (3 extractors run in parallel, one compiler) · **Stack:** Python, LangChain (LCEL, `RunnableParallel`), OpenAI SDK, Pydantic · **Level:** Intermediate

Tagline: Drop in a meeting transcript, get back three things a human would otherwise dig out by hand — the decisions, the action items (with owners), and a short summary — each produced by its own specialist, all run at once.

Charter (what this project is)

After every meeting someone has to write up "what did we decide, who's doing what, and what happened" — and usually nobody does. **MeetingMind** reads a transcript and produces all three: a **decisions** list, an **action items** list (each with an owner), and a short **summary**. Each is a narrow specialist with its own prompt and its own typed output; they run **in parallel** (`RunnableParallel`) because none depends on the others, and a deterministic **compiler** stitches them into one report.

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3

The Story — what you're building

A transcript comes in. Three specialists each read it for one thing: an **action-item extractor** returns a typed list of `{task, owner}`; a **decisions extractor** returns a typed list of decision strings; a **summarizer** returns a few sentences. Because none of them needs another's output, they run at the same time with `RunnableParallel` — faster than one after another. A deterministic **compiler** then folds the three results into one `MeetingReport` and formats it for a human.

The lesson: a lot of "AI for meetings" is really *several narrow extractions*, not one giant prompt that tries to do everything and does each part badly. Give each job its own specialist and its own typed shape, run the independent ones in parallel, and compile — the same supervisor-of-specialists idea as the big multi-agent projects, in its simplest, most useful form.

Project Structure (the professional layout)

`main.py` at the repo root takes a transcript file; all code in `src/`.

```
<repo folder>/ (the MeetingMind repo — full name in the heading above)
├── samples/
│   └── standup.txt             a short sample transcript
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── actions.py          extract_actions chain      (Step 2)
│   │   ├── decisions.py        extract_decisions chain    (Step 2)
│   │   └── summary.py          summarize chain            (Step 2)
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── pipeline.py             run_meeting(), RunnableParallel (Step 3)
│   ├── report.py               compile_report(), format_report() (Step 1)
│   └── schemas.py              ActionItem, Decisions, MeetingReport (Step 1)
├── tests/
│   ├── integration/test_live_meeting.py
│   └── unit/                   test_config, test_report
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

How the structure grows, step by step

- **Step 1** builds the standard setup, `src/schemas.py` and `src/report.py` (the deterministic compiler + formatter) with tests — no model yet.
- **Step 2** adds the three extractor chains in `src/agents/`.
- **Step 3** adds `src/pipeline.py` (`RunnableParallel` + `run_meeting`), `main.py`, and the integration test.

Setup (do this once, before Step 1)

```
cd MeetingMind-Transcript-To-Decisions-Actions-And-Summary
python -m venv .venv
```

```
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install langchain-core langchain-openai openai pydantic python-dotenv
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. Every folder under `src/` gets an empty `__init__.py`. Write one short `samples/standup.txt` transcript (a few lines with a clear decision and an action or two). `LICENSE` is the MIT text.

How testing works in this project

Two layers, one style (plain OK/WRONG scripts; this builds on Doc05, not pytest):

- **`tests/unit/`** — no model: `test_config`; `test_report` (the compiler and formatter, with hand-made parts — a report with 2 decisions and 2 actions formats correctly; an unassigned action reads "unassigned").
- **`tests/integration/test_live_meeting.py`** — the real model: a transcript with a clear decision and action yields a report that names them.

Each step ends with a **Tests for this step** box.

Step 1 — The Report Shape and the Compiler

What this step does: the typed shapes (`ActionItem`, `Decisions`, `MeetingReport`) and the deterministic `compile_report` / `format_report` — built and tested with hand-made parts, no model.

Why this step matters: the compiler is what turns three separate results into one useful report. Proving it with hand-made parts means Step 3 only has to feed it real data, not debug the formatting too.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write the standard setup (`config` / `exceptions` / `utils/logger` / `models/llm`). 2. Write `src/schemas.py`: `ActionItem(task, owner)` (owner optional), `ActionItems(items)`, `Decisions(decisions)`, and `MeetingReport(summary, decisions, action_items)`. 3. Write `src/report.py`: `compile_report(parts)` (take a dict with `summary` / `decisions` / `actions` and build a `MeetingReport`) and `format_report(report)` (a readable block — an action with no owner reads "unassigned").

Tests for this step: `test_report` — a hand-made report formats with its decisions and actions; an owner-less action shows "unassigned". `test_config`.

Step 2 — The Three Extractor Agents

What this step does: the three specialist chains — actions, decisions, summary — each an LCEL chain over the transcript.

Why this step matters: these are the narrow specialists. Two return typed objects (`with_structured_output`); the summarizer returns plain text (`StrOutputParser`). Each has one job and one prompt.

What's new vs. Step 1: `src/agents/*` . **What stays the same:** the schemas and compiler.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. `src/agents/actions.py` : `build_actions_chain(model)` = prompt | `model.with_structured_output(ActionItems)` . 2. `src/agents/decisions.py` : `build_decisions_chain(model)` = prompt | `model.with_structured_output(Decisions)` . 3. `src/agents/summary.py` : `build_summary_chain(model)` = prompt | model | `StrOutputParser()` .

Tests for this step: proven live in Step 3 (they need a model); for now, each `build_*_chain` returns a runnable you can `.invoke({"transcript": ...})` .

Step 3 — Run Them in Parallel and Compile (final)

What this step does: `RunnableParallel` to run all three chains at once, `run_meeting` to compile the results, `main.py` , and the integration test.

Why this step matters: the three jobs are independent, so running them one after another wastes time — `RunnableParallel` runs them together and hands you one dict of results, which the Step 1 compiler turns into the report.

What's new vs. Step 2: `src/pipeline.py` , `main.py` . **What stays the same:** the agents, schemas, compiler.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `src/pipeline.py` : build a `RunnableParallel(summary=..., decisions=..., actions=...)` from the three chains, and `run_meeting(model, transcript)` that invokes it and calls `compile_report` . 2. Write `main.py` (read a transcript file, print `format_report`) and `tests/integration/test_live_meeting.py` .

Final Deliverable: a system that turns a meeting transcript into one report — decisions, action items with owners, and a summary — with the three extractions running in parallel.

Tests for this step: `test_live_meeting` — a transcript with a clear decision and action produces a report that names both.

Run, Test and Configure the Final Project

```
python main.py samples/standup.txt
python -m tests.unit.test_config
python -m tests.unit.test_report
python -m tests.integration.test_live_meeting
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity

Checklist Before You Call This Done

- [ ] Three specialist chains, each with one job and one prompt
- [ ] Two return typed objects; the summarizer returns plain text
- [ ] The three run in parallel (`RunnableParallel`), not one after another
- [ ] The compiler is deterministic and tested with no model
- [ ] An action item with no named owner reads "unassigned", not blank

Publish It to GitHub

Name the repo `MeetingMind-Transcript-To-Decisions-Actions-And-Summary` , add the `README.md` , push. "Turn a transcript into decisions + actions + summary" is a demo everyone instantly understands.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** before the solution. Build the compiler (Step 1) first — it's the part you can fully test.